

=====

CAPSTONE PROJECT

NOTEBOOK 44: FULL-161 TARGETED RARE-TAIL BATCH DEMO

=====

Purpose:

This notebook demonstrates the final full-161 pipeline on

tracks that are known to contain hierarchy-fallback rare-tail

labels, so Stage 2 can be shown properly.

The goal is to:

1. Load the frozen Stage-1 benchmark model
2. Load the Stage-2 rare-tail router
3. Build a targeted batch of test tracks with true rare-tail labels

4. Run the full-161 pipeline on those tracks

5. Compare Stage-2 suggestions with true rare-tail labels

6. Save report-ready rare-tail demo tables

=====

In [1]:

```
# =====
# 1. IMPORT LIBRARIES
# =====

import os
import json
import warnings
from pathlib import Path

import joblib
import librosa
import numpy as np
import pandas as pd
import tensorflow as tf

from scipy.stats import skew, kurtosis

warnings.filterwarnings("ignore", category=FutureWarning)
warnings.filterwarnings("ignore", category=UserWarning)

SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)

print("Seed set to:", SEED)
print("TensorFlow version:", tf.__version__)
```

Seed set to: 42
TensorFlow version: 2.20.0

In [2]:

```
# =====
# 2. LOAD FROZEN ARTIFACTS
# =====

structured_model =
```

```

joblib.load("../models/final_structured_multilabel_candidate150_best_model.joblib")
structured_scaler =
joblib.load("../models/final_structured_multilabel_candidate150_scaler.joblib")
audio_model =
tf.keras.models.load_model("../models/audio_multilabel_candidate150_expanded_final.
keras")

candidate_label_cols = np.load(
    "../data/processed/hybrid_multilabel_candidate150_expanded_label_columns.npy",
    allow_pickle=True
)

stage1_config = {}
with
open("../data/processed/hybrid_multilabel_candidate150_expanded_best_config.txt",
"r") as f:
    for line in f:
        line = line.strip()
        if "=" in line:
            k, v = line.split("=")
            stage1_config[k.strip()] = float(v.strip())

STAGE1_STRUCTURED_WEIGHT = stage1_config["structured_weight"]
STAGE1_AUDIO_WEIGHT = stage1_config["audio_weight"]
STAGE1_THRESHOLD = stage1_config["threshold"]

rare_tail_router_df =
pd.read_csv("../data/processed/full161_rare_tail_routing_table.csv")
genre_inventory_df = pd.read_csv("../data/processed/full_genre_inventory.csv")
full_master_df = pd.read_csv("../data/processed/multilabel_full_master_table.csv")

stage2_config = {}
with open("../data/processed/full161_stage2_best_config.txt", "r") as f:
    for line in f:
        line = line.strip()
        if "=" in line:
            k, v = line.split("=")
            stage2_config[k.strip()] = v.strip()

STAGE2_ANCHOR_TRIGGER_THRESHOLD = float(stage2_config["anchor_trigger_threshold"])
STAGE2_TOP_K = int(stage2_config["top_k"])

features_reference = pd.read_csv(
    "../data/raw/metadata/features.csv",
    header=[0, 1, 2],
    index_col=0
)

print("Structured model loaded.")
print("Audio model loaded.")
print("Candidate labels:", len(candidate_label_cols))
print("Stage-1 config:", stage1_config)
print("Stage-2 config:", stage2_config)
print("Rare-tail router shape:", rare_tail_router_df.shape)
print("Full master shape:", full_master_df.shape)
print("Reference features shape:", features_reference.shape)

```

```

Structured model loaded.
Audio model loaded.
Candidate labels: 150
Stage-1 config: {'structured_weight': 0.1, 'audio_weight': 0.9, 'threshold': 0.2}
Stage-2 config: {'anchor_trigger_threshold': '0.1', 'top_k': '1'}
Rare-tail router shape: (13, 26)
Full master shape: (81574, 170)
Reference features shape: (106574, 518)

```

In [3]:

```

# =====
# 3. PREPARE LOOKUPS
# =====

genre_inventory_df["genre_id"] = genre_inventory_df["genre_id"].astype(int)
genre_name_map = dict(zip(genre_inventory_df["genre_id"],
                           genre_inventory_df["genre_name"]))

candidate_label_ids = [int(col.replace("genre_", "")) for col in
                        candidate_label_cols]
candidate_id_to_index = {int(col.replace("genre_", "")): i for i, col in
                          enumerate(candidate_label_cols)}

fallback_router_df = rare_tail_router_df[
    rare_tail_router_df["fallback_mode"] == "Hierarchy-triggered fallback"
].copy().reset_index(drop=True)

fallback_router_df["rare_tail_genre_id"] =
fallback_router_df["rare_tail_genre_id"].astype(int)
fallback_router_df["anchor_candidate_id"] =
fallback_router_df["anchor_candidate_id"].astype(int)

inventory_only_df = rare_tail_router_df[
    rare_tail_router_df["fallback_mode"] == "Inventory only"
].copy().reset_index(drop=True)

features_reference.columns = [
    "_".join([str(level) for level in col]).strip()
    for col in features_reference.columns.to_flat_index()
]

reference_feature_df = features_reference.copy()
reference_feature_df = reference_feature_df.select_dtypes(include=["number"])
reference_feature_df = reference_feature_df.replace([np.inf, -np.inf], np.nan)
reference_feature_means = reference_feature_df.mean(axis=0)
reference_feature_columns = list(reference_feature_df.columns)

full_master_indexed = full_master_df.set_index("track_id", drop=False)

fallback_rare_ids = fallback_router_df["rare_tail_genre_id"].astype(int).tolist()
fallback_rare_cols = [f"genre_{gid}" for gid in fallback_rare_ids]

print("Fallback rare-tail labels:", len(fallback_rare_ids))
print("Inventory-only rare-tail labels:", inventory_only_df.shape[0])

```

```
print("Structured reference feature columns:", len(reference_feature_columns))
display(fallback_router_df)
```

Fallback rare-tail labels: 10
Inventory-only rare-tail labels: 3
Structured reference feature columns: 518

	rare_tail_genre_id	rare_tail_genre_name	parent_id	parent_name	root_genre_id	root_genre_
0	176	Pacific	2	International	2	Interna
1	1060	Tango	46	Latin America	2	Interna
2	465	Musical Theater	20	Spoken	20	Sp
3	189	Talk Radio	65	Radio	20	Sp
4	1032	Turkish	102	Middle East	2	Interna
5	374	Banter	20	Spoken	20	Sp
6	173	N. Indian Traditional	86	Indian	2	Interna
7	493	Western Swing	651	Country & Western	9	Co
8	377	Deep Funk	19	Funk	14	Sou
9	808	Salsa	46	Latin America	2	Interna

10 rows × 26 columns



In [4]:

```
# =====
# 4. BUILD TARGETED RARE-TAIL TEST COHORT
# =====
```

```

target_df = full_master_df.copy()

target_df["true_fallback_rare_tail_count"] =
target_df[fallback_rare_cols].sum(axis=1)

targeted_test_df = target_df[
    (target_df["split"] == "test") &
    (target_df["true_fallback_rare_tail_count"] > 0)
].copy().reset_index(drop=True)

print("Targeted rare-tail test cohort shape:", targeted_test_df.shape)

print("Targeted rare-tail test cohort preview:")
display(
    targeted_test_df[
        ["track_id", "title", "audio_path", "true_fallback_rare_tail_count"]
    ].head(20)
)

# Label distribution in the targeted cohort
rare_tail_distribution_rows = []
for gid in fallback_rare_ids:
    col = f"genre_{gid}"
    rare_tail_distribution_rows.append({
        "rare_tail_genre_id": gid,
        "rare_tail_genre_name": genre_name_map.get(gid, str(gid)),
        "targeted_test_count": int(targeted_test_df[col].sum())
    })

rare_tail_distribution_df = pd.DataFrame(rare_tail_distribution_rows).sort_values(
    ["targeted_test_count", "rare_tail_genre_name"],
    ascending=[False, True]
).reset_index(drop=True)

print("Rare-tail label distribution in targeted cohort:")
display(rare_tail_distribution_df)

```

Targeted rare-tail test cohort shape: (35, 171)

Targeted rare-tail test cohort preview:

	track_id	title	audio_path	true_fallback_rare_tail_cou
0	16930	Open Session	../data/raw/audio/fma_large\016\016930.mp3	
1	30180	Conic Sections	../data/raw/audio/fma_large\030\030180.mp3	
2	30181	Elliptically	../data/raw/audio/fma_large\030\030181.mp3	
3	30182	Hyperboli	../data/raw/audio/fma_large\030\030182.mp3	
4	30183	Directrix	../data/raw/audio/fma_large\030\030183.mp3	
5	30184	So It Goes	../data/raw/audio/fma_large\030\030184.mp3	
6	30185	Quitting Time	../data/raw/audio/fma_large\030\030185.mp3	
7	30186	Roundabout	../data/raw/audio/fma_large\030\030186.mp3	
8	30187	Inscribed Pythagorus	../data/raw/audio/fma_large\030\030187.mp3	
9	30188	The Electricity Song	../data/raw/audio/fma_large\030\030188.mp3	
10	30189	Unit Circle Trigonometry	../data/raw/audio/fma_large\030\030189.mp3	
11	35853	Untitled	../data/raw/audio/fma_large\035\035853.mp3	
12	35854	Untitled	../data/raw/audio/fma_large\035\035854.mp3	
13	35857	Fanfare for the King of Tonga	../data/raw/audio/fma_large\035\035857.mp3	
14	35858	The King's Swim Fins	../data/raw/audio/fma_large\035\035858.mp3	
15	51013	The City	../data/raw/audio/fma_large\051\051013.mp3	
16	51014	Black-Bar Trombone	../data/raw/audio/fma_large\051\051014.mp3	
17	99351	Ille de Roman Olsun	../data/raw/audio/fma_large\099\099351.mp3	
18	99352	Osman Aga	../data/raw/audio/fma_large\099\099352.mp3	
19	99353	Zeytinyagli	../data/raw/audio/fma_large\099\099353.mp3	



Rare-tail label distribution in targeted cohort:

	rare_tail_genre_id	rare_tail_genre_name	targeted_test_count
0	1060	Tango	12
1	465	Musical Theater	10
2	1032	Turkish	5
3	176	Pacific	4
4	493	Western Swing	2
5	173	N. Indian Traditional	1
6	189	Talk Radio	1
7	374	Banter	0
8	377	Deep Funk	0
9	808	Salsa	0

In [5]:

```

# =====
# 5. AUDIO / FEATURE SETTINGS
# =====

SR = 22050
DURATION = 15
N_MELS = 64
N_FFT = 2048
HOP_LENGTH = 1024
MAX_FRAMES = int(np.ceil((DURATION * SR) / HOP_LENGTH)) + 1

print("SR:", SR)
print("DURATION:", DURATION)
print("N_MELS:", N_MELS)
print("MAX_FRAMES:", MAX_FRAMES)

```

```

SR: 22050
DURATION: 15
N_MELS: 64
MAX_FRAMES: 324

```

In [6]:

```

# =====
# 6. HELPER FUNCTIONS
# =====

def load_audio(file_path, sr=SR, duration=DURATION):
    y, sr_loaded = librosa.load(file_path, sr=sr, mono=True, duration=duration)
    if y is None or len(y) == 0:
        raise ValueError(f"Could not load usable audio from: {file_path}")
    return y, sr_loaded

```



```

def build_mel_input(y, sr=SR, n_mels=N_MELS, n_fft=N_FFT, hop_length=HOP_LENGTH,
max_frames=MAX_FRAMES):
    mel = librosa.feature.melspectrogram(
        y=y,
        sr=sr,
        n_fft=n_fft,
        hop_length=hop_length,
        n_mels=n_mels
    )
    mel_db = librosa.power_to_db(mel, ref=np.max)
    mel_db = np.clip((mel_db + 80.0) / 80.0, 0.0, 1.0)

    if mel_db.shape[1] < max_frames:
        pad_width = max_frames - mel_db.shape[1]
        mel_db = np.pad(mel_db, ((0, 0), (0, pad_width)), mode="constant")
    else:
        mel_db = mel_db[:, :max_frames]

    return mel_db.astype(np.float32)[None, :, :, None]

def safe_stat_vector(arr_2d, stat_name):
    arr_2d = np.asarray(arr_2d, dtype=np.float64)

    if arr_2d.ndim == 1:
        arr_2d = arr_2d.reshape(1, -1)

    if stat_name == "mean":
        out = np.mean(arr_2d, axis=1)
    elif stat_name == "std":
        out = np.std(arr_2d, axis=1)
    elif stat_name == "median":
        out = np.median(arr_2d, axis=1)
    elif stat_name == "min":
        out = np.min(arr_2d, axis=1)
    elif stat_name == "max":
        out = np.max(arr_2d, axis=1)
    elif stat_name == "skew":
        out = skew(arr_2d, axis=1, bias=False, nan_policy="omit")
    elif stat_name == "kurtosis":
        out = kurtosis(arr_2d, axis=1, bias=False, nan_policy="omit")
    else:
        raise ValueError(f"Unknown stat: {stat_name}")

    out = np.asarray(out, dtype=np.float64)
    out[~np.isfinite(out)] = 0.0
    return out.astype(np.float32)

def build_feature_matrices(y, sr=SR):
    y = np.asarray(y, dtype=np.float64)
    y_harmonic = librosa.effects.harmonic(y)

    mats = {}
    mats["chroma_stft"] = librosa.feature.chroma_stft(y=y, sr=sr)
    mats["chroma_cqt"] = librosa.feature.chroma_cqt(y=y, sr=sr)
    mats["chroma_cens"] = librosa.feature.chroma_cens(y=y, sr=sr)
    mats["tonnetz"] = librosa.feature.tonnetz(y=y_harmonic, sr=sr)
    mats["mfcc"] = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=20)

```

```

mats["rms"] = librosa.feature.rms(y=y)
mats["spectral_centroid"] = librosa.feature.spectral_centroid(y=y, sr=sr)
mats["spectral_bandwidth"] = librosa.feature.spectral_bandwidth(y=y, sr=sr)
mats["spectral_contrast"] = librosa.feature.spectral_contrast(y=y, sr=sr)
mats["spectral_rolloff"] = librosa.feature.spectral_rolloff(y=y, sr=sr)
mats["zcr"] = librosa.feature.zero_crossing_rate(y)

return mats

def build_structured_feature_vector(y, reference_columns, reference_means, sr=SR):
    feature_mats = build_feature_matrices(y, sr=sr)
    row_dict = {}

    for col in reference_columns:
        parts = col.split("_")
        component_idx = int(parts[-1]) - 1
        stat_name = parts[-2]
        feature_name = "_".join(parts[:-2])

        if feature_name in feature_mats:
            mat = feature_mats[feature_name]
            stat_vec = safe_stat_vector(mat, stat_name)

            if 0 <= component_idx < len(stat_vec):
                row_dict[col] = float(stat_vec[component_idx])
            else:
                row_dict[col] = np.nan
        else:
            row_dict[col] = np.nan

    X_one = pd.DataFrame([row_dict], columns=reference_columns)
    X_one = X_one.replace([np.inf, -np.inf], np.nan)

    for col in reference_columns:
        if pd.isna(X_one.loc[0, col]):
            X_one.loc[0, col] = float(reference_means[col])

    return X_one.astype(np.float32)

def scores_to_pseudoprobs(score_matrix):
    clipped = np.clip(score_matrix, -20, 20)
    return 1.0 / (1.0 + np.exp(-clipped))

def get_structured_scores(model, X_scaled):
    if hasattr(model, "predict_proba"):
        scores = model.predict_proba(X_scaled)
    elif hasattr(model, "decision_function"):
        scores = model.decision_function(X_scaled)
    else:
        raise ValueError("Structured model supports neither predict_proba nor decision_function.")
    return np.asarray(scores)

def fuse_probabilities(structured_probs, audio_probs, w_structured, w_audio):
    return (w_structured * structured_probs) + (w_audio * audio_probs)

def decode_stage1(prob_matrix, threshold):

```

```

    return (prob_matrix >= threshold).astype(np.uint8)

def build_rare_tail_scores_for_single(stage1_probs, router_df,
candidate_index_map):
    scores = []

    for _, row in router_df.iterrows():
        anchor_id = int(row["anchor_candidate_id"])
        anchor_idx = candidate_index_map[anchor_id]
        anchor_prob = float(stage1_probs[0, anchor_idx])

        p_anchor = 0.0 if pd.isna(row["p_rare_given_anchor"]) else
float(row["p_rare_given_anchor"])
        p_root = 0.0 if pd.isna(row["p_rare_given_root"]) else
float(row["p_rare_given_root"])

        strength = max(p_anchor, p_root)
        rare_score = anchor_prob * strength

        scores.append({
            "rare_tail_genre_id": int(row["rare_tail_genre_id"]),
            "rare_tail_genre_name": row["rare_tail_genre_name"],
            "anchor_candidate_id": anchor_id,
            "anchor_candidate_name": row["anchor_candidate_name"],
            "anchor_prob": anchor_prob,
            "rare_tail_score": rare_score,
            "fallback_mode": row["fallback_mode"]
        })

    return pd.DataFrame(scores)

def get_true_labels(track_id):
    row = full_master_indexed.loc[track_id]

    true_candidate_ids = []
    for col in candidate_label_cols:
        if int(row[col]) == 1:
            true_candidate_ids.append(int(col.replace("genre_", "")))

    true_rare_ids = []
    for col in fallback_rare_cols:
        if col in row.index and int(row[col]) == 1:
            true_rare_ids.append(int(col.replace("genre_", "")))

    return (
        true_candidate_ids,
        [genre_name_map.get(gid, str(gid)) for gid in true_candidate_ids],
        true_rare_ids,
        [genre_name_map.get(gid, str(gid)) for gid in true_rare_ids],
    )

```

In [7]:

```

# =====
# 7. RUN FULL-161 PIPELINE ACROSS TARGETED RARE-TAIL TRACKS

```

```

# =====

targeted_rows = []
problem_files = []

for _, row in targeted_test_df.iterrows():
    track_id = int(row["track_id"])
    audio_path = row["audio_path"]
    title = row.get("title", None)

    try:
        y, sr_loaded = load_audio(audio_path, sr=SR, duration=DURATION)

        X_audio_input = build_mel_input(
            y,
            sr=sr_loaded,
            n_mels=N_MELS,
            n_fft=N_FFT,
            hop_length=HOP_LENGTH,
            max_frames=MAX_FRAMES
        )

        X_structured_one = build_structured_feature_vector(
            y,
            reference_feature_columns,
            reference_feature_means,
            sr=sr_loaded
        )

        X_structured_scaled =
structured_scaler.transform(X_structured_one).astype(np.float32)

        structured_scores = get_structured_scores(structured_model,
X_structured_scaled)
        structured_probs = scores_to_pseudoprobs(structured_scores)

        audio_probs = audio_model.predict(X_audio_input, verbose=0)

        stage1_probs = fuse_probabilities(
            structured_probs,
            audio_probs,
            STAGE1_STRUCTURED_WEIGHT,
            STAGE1_AUDIO_WEIGHT
        )

        stage1_pred = decode_stage1(stage1_probs, STAGE1_THRESHOLD)

        predicted_candidate_ids = [
            int(col.replace("genre_", ""))
            for j, col in enumerate(candidate_label_cols)
            if int(stage1_pred[0, j]) == 1
        ]
        predicted_candidate_names = [genre_name_map.get(gid, str(gid)) for gid in
predicted_candidate_ids]

        rare_tail_scores_df = build_rare_tail_scores_for_single(
            stage1_probs,

```

```

        fallback_router_df,
        candidate_id_to_index
    ).sort_values(["rare_tail_score", "anchor_prob"],
ascending=False).reset_index(drop=True)

    stage2_suggestions_df = rare_tail_scores_df[
        rare_tail_scores_df["anchor_prob"] >= STAGE2_ANCHOR_TRIGGER_THRESHOLD
    ].head(STAGE2_TOP_K).copy()

    rare_tail_ids =
stage2_suggestions_df["rare_tail_genre_id"].astype(int).tolist() if
len(stage2_suggestions_df) > 0 else []
    rare_tail_names = stage2_suggestions_df["rare_tail_genre_name"].tolist() if
len(stage2_suggestions_df) > 0 else []
    rare_tail_scores =
stage2_suggestions_df["rare_tail_score"].round(6).tolist() if
len(stage2_suggestions_df) > 0 else []

    true_candidate_ids, true_candidate_names, true_rare_ids, true_rare_names =
get_true_labels(track_id)

    candidate_hit_any = any(gid in predicted_candidate_ids for gid in
true_candidate_ids) if len(true_candidate_ids) > 0 else False
    rare_tail_hit_any = any(gid in rare_tail_ids for gid in true_rare_ids) if
len(true_rare_ids) > 0 else False

    rare_tail_hit_exact_top1 = False
    if len(true_rare_ids) > 0 and len(rare_tail_ids) > 0:
        rare_tail_hit_exact_top1 = rare_tail_ids[0] in true_rare_ids

    targeted_rows.append({
        "track_id": track_id,
        "title": title,
        "audio_path": audio_path,
        "true_candidate_label_count": len(true_candidate_ids),
        "true_candidate_label_names": true_candidate_names,
        "predicted_candidate_label_count": len(predicted_candidate_ids),
        "predicted_candidate_label_names": predicted_candidate_names,
        "candidate_hit_any": candidate_hit_any,
        "true_rare_tail_label_count": len(true_rare_ids),
        "true_rare_tail_label_names": true_rare_names,
        "stage2_rare_tail_suggestion_count": len(rare_tail_ids),
        "stage2_rare_tail_suggestion_names": rare_tail_names,
        "stage2_rare_tail_scores": rare_tail_scores,
        "rare_tail_hit_any": rare_tail_hit_any,
        "rare_tail_hit_exact_top1": rare_tail_hit_exact_top1
    })

    except Exception as e:
        problem_files.append({
            "track_id": track_id,
            "title": title,
            "audio_path": audio_path,
            "error": str(e)
        })

    targeted_results_df = pd.DataFrame(targeted_rows)

```

```
print("Targeted rare-tail batch results shape:", targeted_results_df.shape)
print("Problem files:", len(problem_files))
display(targeted_results_df)
```

Targeted rare-tail batch results shape: (35, 15)
Problem files: 0

	track_id	title	audio_path	true_candidate_label_co
0	16930	Open Session	../data/raw/audio/fma_large\016\016930.mp3	
1	30180	Conic Sections	../data/raw/audio/fma_large\030\030180.mp3	
2	30181	Elliptically	../data/raw/audio/fma_large\030\030181.mp3	
3	30182	Hyperboli	../data/raw/audio/fma_large\030\030182.mp3	
4	30183	Directrix	../data/raw/audio/fma_large\030\030183.mp3	
5	30184	So It Goes	../data/raw/audio/fma_large\030\030184.mp3	
6	30185	Quitting Time	../data/raw/audio/fma_large\030\030185.mp3	
7	30186	Roundabout	../data/raw/audio/fma_large\030\030186.mp3	
8	30187	Inscribed Pythagorus	../data/raw/audio/fma_large\030\030187.mp3	
9	30188	The Electricity Song	../data/raw/audio/fma_large\030\030188.mp3	
10	30189	Unit Circle Trigonometry	../data/raw/audio/fma_large\030\030189.mp3	
11	35853	Untitled	../data/raw/audio/fma_large\035\035853.mp3	
12	35854	Untitled	../data/raw/audio/fma_large\035\035854.mp3	
13	35857	Fanfare for the King of Tonga	../data/raw/audio/fma_large\035\035857.mp3	
14	35858	The King's Swim Fins	../data/raw/audio/fma_large\035\035858.mp3	
15	51013	The City	../data/raw/audio/fma_large\051\051013.mp3	
16	51014	Black-Bar Trombone	../data/raw/audio/fma_large\051\051014.mp3	
17	99351	Ille de Roman Olsun	../data/raw/audio/fma_large\099\099351.mp3	
18	99352	Osman Aga	../data/raw/audio/fma_large\099\099352.mp3	

	track_id	title	audio_path	true_candidate_label_co
19	99353	Zeytinyagli	../data/raw/audio/fma_large\099\099353.mp3	
20	99354	Sevdam Olmasa / Muhabbet Bagi	../data/raw/audio/fma_large\099\099354.mp3	
21	99355	Kim Arar	../data/raw/audio/fma_large\099\099355.mp3	
22	108156	Río Rivadavia	../data/raw/audio/fma_large\108\108156.mp3	
23	108157	Autoayuda	../data/raw/audio/fma_large\108\108157.mp3	
24	108158	Spring inside	../data/raw/audio/fma_large\108\108158.mp3	
25	108159	Nocturno #2	../data/raw/audio/fma_large\108\108159.mp3	
26	108160	Aloe	../data/raw/audio/fma_large\108\108160.mp3	
27	108161	Pasará	../data/raw/audio/fma_large\108\108161.mp3	
28	108162	Fantasia en G para un pueblo al sur del mapa	../data/raw/audio/fma_large\108\108162.mp3	
29	108163	Trovadora del Viento	../data/raw/audio/fma_large\108\108163.mp3	
30	108164	Por mí, que llueva	../data/raw/audio/fma_large\108\108164.mp3	
31	108165	Hace frío (en NYC)	../data/raw/audio/fma_large\108\108165.mp3	
32	108166	Verde	../data/raw/audio/fma_large\108\108166.mp3	
33	108167	Fruto prohibido	../data/raw/audio/fma_large\108\108167.mp3	
34	146485	Live on WFMU at Monty Hall/Transpacific Sound ...	../data/raw/audio/fma_large\146\146485.mp3	

In [8]:

```
# =====  
# 8. BUILD TARGETED RARE-TAIL SUMMARY
```



```
# =====

summary_rows = []

n_tracks = len(targeted_results_df)
candidate_hits = int(targeted_results_df["candidate_hit_any"].sum()) if n_tracks > 0 else 0
rare_tail_hits_any = int(targeted_results_df["rare_tail_hit_any"].sum()) if n_tracks > 0 else 0
rare_tail_hits_top1 = int(targeted_results_df["rare_tail_hit_exact_top1"].sum()) if n_tracks > 0 else 0
tracks_with_any_stage2 = int((targeted_results_df["stage2_rare_tail_suggestion_count"] > 0).sum()) if n_tracks > 0 else 0

summary_rows.append({
    "Metric": "Tracks in targeted rare-tail batch",
    "Value": n_tracks
})
summary_rows.append({
    "Metric": "Candidate hit-any count",
    "Value": candidate_hits
})
summary_rows.append({
    "Metric": "Candidate hit-any rate",
    "Value": (candidate_hits / n_tracks) if n_tracks > 0 else np.nan
})
summary_rows.append({
    "Metric": "Tracks with any Stage-2 suggestion",
    "Value": tracks_with_any_stage2
})
summary_rows.append({
    "Metric": "Stage-2 suggestion coverage",
    "Value": (tracks_with_any_stage2 / n_tracks) if n_tracks > 0 else np.nan
})
summary_rows.append({
    "Metric": "Rare-tail hit-any count",
    "Value": rare_tail_hits_any
})
summary_rows.append({
    "Metric": "Rare-tail hit-any rate",
    "Value": (rare_tail_hits_any / n_tracks) if n_tracks > 0 else np.nan
})
summary_rows.append({
    "Metric": "Rare-tail exact top-1 hit count",
    "Value": rare_tail_hits_top1
})
summary_rows.append({
    "Metric": "Rare-tail exact top-1 hit rate",
    "Value": (rare_tail_hits_top1 / n_tracks) if n_tracks > 0 else np.nan
})
summary_rows.append({
    "Metric": "Average predicted candidate labels",
    "Value": float(targeted_results_df["predicted_candidate_label_count"].mean()) if n_tracks > 0 else np.nan
})
summary_rows.append({
```

```

    "Metric": "Average Stage-2 rare-tail suggestions",
    "Value": float(targeted_results_df["stage2_rare_tail_suggestion_count"].mean())
if n_tracks > 0 else np.nan
}))

targeted_summary_df = pd.DataFrame(summary_rows)

print("Targeted rare-tail batch summary:")
display(targeted_summary_df)

```

Targeted rare-tail batch summary:

	Metric	Value
0	Tracks in targeted rare-tail batch	35.000000
1	Candidate hit-any count	30.000000
2	Candidate hit-any rate	0.857143
3	Tracks with any Stage-2 suggestion	23.000000
4	Stage-2 suggestion coverage	0.657143
5	Rare-tail hit-any count	1.000000
6	Rare-tail hit-any rate	0.028571
7	Rare-tail exact top-1 hit count	1.000000
8	Rare-tail exact top-1 hit rate	0.028571
9	Average predicted candidate labels	4.342857
10	Average Stage-2 rare-tail suggestions	0.657143

In [9]:

```

# =====
# 9. BUILD RARE-TAIL LABEL-LEVEL SUMMARY
# =====

label_level_rows = []

for gid in fallback_rare_ids:
    gname = genre_name_map.get(gid, str(gid))

    true_mask = targeted_results_df["true_rare_tail_label_names"].apply(lambda x:
gname in x if isinstance(x, list) else False)
    sugg_mask =
targeted_results_df["stage2_rare_tail_suggestion_names"].apply(lambda x: gname in x
if isinstance(x, list) else False)

    true_count = int(true_mask.sum())
    suggested_count = int(sugg_mask.sum())

    hit_count = 0
    if true_count > 0:

```

```

hit_count = int(((true_mask) & (sugg_mask)).sum())

label_level_rows.append({
    "rare_tail_genre_id": gid,
    "rare_tail_genre_name": gname,
    "true_track_count": true_count,
    "suggested_track_count": suggested_count,
    "hit_count": hit_count,
    "label_recall": (hit_count / true_count) if true_count > 0 else np.nan
})

rare_tail_label_level_df = pd.DataFrame(label_level_rows).sort_values(
    ["true_track_count", "hit_count", "rare_tail_genre_name"],
    ascending=[False, False, True]
).reset_index(drop=True)

print("Rare-tail label-level summary:")
display(rare_tail_label_level_df)

```

Rare-tail label-level summary:

	rare_tail_genre_id	rare_tail_genre_name	true_track_count	suggested_track_count	hit_count
0	1060	Tango	12	2	1
1	465	Musical Theater	10	0	0
2	1032	Turkish	5	3	0
3	176	Pacific	4	12	0
4	493	Western Swing	2	0	0
5	173	N. Indian Traditional	1	0	0
6	189	Talk Radio	1	0	0
7	374	Banter	0	5	0
8	377	Deep Funk	0	1	0
9	808	Salsa	0	0	0

In [10]:

```

# =====
# 10. BUILD HUMAN-READABLE DEMO TABLE
# =====

demo_columns = [
    "track_id",
    "title",
    "true_candidate_label_names",
    "predicted_candidate_label_names",
    "candidate_hit_any",
    "true_rare_tail_label_names",

```

```
"stage2_rare_tail_suggestion_names",  
"stage2_rare_tail_scores",  
"rare_tail_hit_any",  
"rare_tail_hit_exact_top1"  
]  
  
targeted_demo_df = targeted_results_df[demo_columns].copy()  
  
print("Targeted rare-tail demo table:")  
display(targeted_demo_df)
```

Targeted rare-tail demo table:

	track_id	title	true_candidate_label_names	predicted_candidate_label_names	car
0	16930	Open Session	[International, Spoken, Radio, Middle East]	[Rock, Experimental, Improv]	
1	30180	Conic Sections	[Novelty, Pop, Spoken, Experimental]	[Rock, Folk, Experimental]	
2	30181	Elliptically	[Novelty, Pop, Spoken, Experimental]	[International, Rock, Folk, Experimental]	
3	30182	Hyperboli	[Novelty, Pop, Spoken, Experimental]	[International, Pop, Rock, Folk, Experimental,...]	
4	30183	Directrix	[Novelty, Pop, Spoken, Experimental]	[Rock, Lo-Fi, Experimental]	
5	30184	So It Goes	[Novelty, Pop, Spoken, Experimental]	[International, Blues, Pop, Rock, Folk, Experi...]	
6	30185	Quitting Time	[Novelty, Pop, Spoken, Experimental]	[Avant-Garde, Rock, Folk, Experimental]	
7	30186	Roundabout	[Novelty, Pop, Spoken, Experimental]	[Avant-Garde, Pop, Rock, Electronic, Hip-Hop, ...]	
8	30187	Inscribed Pythagorus	[Novelty, Pop, Spoken, Experimental]	[Pop, Rock, Folk, Hip-Hop, Experimental]	
9	30188	The Electricity Song	[Novelty, Pop, Spoken, Experimental]	[International, Rock, Folk, Experimental]	
10	30189	Unit Circle Trigonometry	[Novelty, Pop, Spoken, Experimental]	[Rock, Folk, Lo-Fi, Experimental]	
11	35853	Untitled	[International, Novelty, Rock, Punk, Experimen...]	[Rock, Folk, Lo-Fi, Experimental]	
12	35854	Untitled	[International, Novelty, Rock, Punk, Experimen...]	[Rock, Folk, Experimental, Instrumental]	
13	35857	Fanfare for the King of Tonga	[International, Novelty, Rock, Punk, Experimen...]	[Rock, Experimental]	
14	35858	The King's Swim Fins	[International, Novelty, Rock, Punk, Experimen...]	[Rock, Experimental, Improv]	
15	51013	The City	[Country, Rock, Ambient, Shoegaze, Country & W...]	[Electronic, Experimental, Instrumental]	
16	51014	Black-Bar Trombone	[Country, Rock, Ambient, Shoegaze, Country & W...]	[Avant-Garde, Rock, Electronic, Noise, Experim...]	
17	99351	Ille de Roman Olsun	[International, Folk, Middle East]	[Avant-Garde, International, Rock, Folk, Exper...]	
18	99352	Osman Aga	[International, Folk, Middle East]	[Avant-Garde, Rock, Folk, Experimental, Improv]	

	track_id	title	true_candidate_label_names	predicted_candidate_label_names	car
19	99353	Zeytinyagli	[International, Folk, Middle East]	[Avant-Garde, International, Rock, Folk, Exper...	
20	99354	Sevdam Olmasa / Muhabbet Bagi	[International, Folk, Middle East]	[Rock, Electronic, Experimental]	
21	99355	Kim Arar	[International, Folk, Middle East]	[Rock, Experimental, Improv]	
22	108156	Río Rivadavia	[International, Folk, Latin America, Europe, S...	[Pop, Rock, Folk, Experimental]	
23	108157	Autoayuda	[International, Folk, Latin America, Europe, S...	[International, Pop, Rock, Folk, Experimental]	
24	108158	Spring inside	[International, Folk, Latin America, Europe, S...	[Rock, Folk, Experimental, Instrumental]	
25	108159	Nocturno #2	[International, Folk, Latin America, Europe, S...	[Rock, Electronic, Folk, Experimental, Instrum...	
26	108160	Aloe	[International, Folk, Latin America, Europe, S...	[Classical, Folk, Experimental, Ambient, Instr...	
27	108161	Pasará	[International, Folk, Latin America, Europe, S...	[Pop, Rock, Folk, Experimental, Experimental P...	
28	108162	Fantasia en G para un pueblo al sur del mapa	[International, Folk, Latin America, Europe, S...	[Pop, Rock, Folk, Instrumental]	
29	108163	Trovadora del Viento	[International, Folk, Latin America, Europe, S...	[Pop, Rock, Lo-Fi, Experimental]	
30	108164	Por mí, que llueva	[International, Folk, Latin America, Europe, S...	[Pop, Rock, Folk, Experimental, Singer-Songwri...	
31	108165	Hace frío (en NYC)	[International, Folk, Latin America, Europe, S...	[International, Pop, Rock, Folk, Experimental]	
32	108166	Verde	[International, Folk, Latin America, Europe, S...	[Rock, Folk, Experimental]	
33	108167	Fruto prohibido	[International, Folk, Latin America, Europe, S...	[Pop, Rock, Folk, Experimental, Singer-Songwri...	
34	146485	Live on WFMU at Monty Hall/Transpacific Sound ...	[International, Indian]	[Avant-Garde, Rock, Electronic, Hip-Hop, Exper...	

In [11]:

```
# =====
# 11. SAVE OUTPUTS
```

```
# =====

os.makedirs("../data/processed", exist_ok=True)

targeted_results_df.to_csv(
    "../data/processed/full161_targeted_rare_tail_batch_results.csv",
    index=False
)

targeted_summary_df.to_csv(
    "../data/processed/full161_targeted_rare_tail_batch_summary.csv",
    index=False
)

rare_tail_label_level_df.to_csv(
    "../data/processed/full161_targeted_rare_tail_label_level_summary.csv",
    index=False
)

targeted_demo_df.to_csv(
    "../data/processed/full161_targeted_rare_tail_demo_table.csv",
    index=False
)

if len(problem_files) > 0:
    pd.DataFrame(problem_files).to_csv(
        "../data/processed/full161_targeted_rare_tail_problem_files.csv",
        index=False
    )

print("Saved full-161 targeted rare-tail batch outputs.")
```

Saved full-161 targeted rare-tail batch outputs.

In [12]:

```
# =====
# 12. INTERPRETATION NOTES
# =====

print("1. This notebook demonstrates the full-161 pipeline on tracks that actually contain hierarchy-fallback rare-tail labels.")
print("2. Stage 1 still produces the main candidate-label predictions.")
print("3. Stage 2 is evaluated here specifically on rare-tail behaviour rather than on a random sample.")
print("4. The exported tables are useful as qualitative evidence for the rare-tail component of the pipeline.")
print("5. This notebook complements Notebook 43 by showing the part that a random batch may miss.")
```

1. This notebook demonstrates the full-161 pipeline on tracks that actually contain hierarchy-fallback rare-tail labels.
2. Stage 1 still produces the main candidate-label predictions.
3. Stage 2 is evaluated here specifically on rare-tail behaviour rather than on a random sample.
4. The exported tables are useful as qualitative evidence for the rare-tail component of the pipeline.
5. This notebook complements Notebook 43 by showing the part that a random batch may miss.